Rendu de monnaie

Nous allons maintenant travailler sur un problème d'optimisation déjà rencontré l'année dernière : le problème du rendu de monnaie.

Petit rappel : vous avez à votre disposition un nombre illimité de pièces de 2 cts, 5 cts, 10 cts, 50 cts et 1 euro (100 cts). Vous devez rendre une certaine somme (rendu de monnaie). Le problème est le suivant : "Quel est le nombre minimum de pièces qui doivent être utilisées pour rendre la monnaie"

La résolution "gloutonne" de ce problème peut être la suivante :

1. on prend la pièce qui a la plus grande valeur (il faut que la valeur de cette pièce soit inférieure ou égale à la somme restant à rendre)
2. on recommence l'opération ci-dessus jusqu'au moment où la somme à rendre est égale à zéro.

Prenons un exemple :

nous avons 1 euro 77 cts à rendre :

- on utilise une pièce de 1 euro (plus grande valeur de pièce inférieure à 1,77 euro), il reste 77 cts à rendre
- on utilise une pièce de 50 cts (plus grande valeur de pièce inférieure à 0,77 euro), il reste 27 cts à rendre
- on utilise une pièce de 10 cts (plus grande valeur de pièce inférieure à 0,27 euro), il reste 17 cts à rendre
- on utilise une pièce de 10 cts (plus grande valeur de pièce inférieure à 0,17 euro), il reste 7 cts à rendre
- on utilise une pièce de 5 cts (plus grande valeur de pièce inférieure à 0,07 euro), il reste 2 cts à rendre
- on utilise une pièce de 2 cts (plus grande valeur de pièce inférieure à 0,02 euro), il reste 0 cts à rendre

L'algorithme se termine en renvoyant 6 (on a dû rendre 6 pièces)

À faire vous-même 2

Appliquez l'algorithme glouton vu ci-dessus avec la somme à rendre égale à 11 centimes.

Comme vous l'avez sans doute remarqué, si on applique l'algorithme glouton à cet exemple, nous ne trouvons pas de réponse. En effet :

- on utilise une pièce de 10 cts (plus grande valeur de pièce inférieure à 11 centimes), il reste 1 cts à rendre
- il n'y a pas de pièce de 1 cts => l'algorithme est "bloqué"

Cet exemple marque une caractéristique importante des algorithmes glouton : une fois qu'une "décision" a été prise, on ne revient pas "en arrière" (on a choisi la pièce de 10 cts, même si cela nous conduit dans une "impasse").

Rappel : dans certains cas, un algorithme glouton trouvera une solution, mais cette dernière ne sera pas "une des meilleures solutions possible" (une solution optimale), voir le cours sur les algorithmes glouton.

Évidemment, le fait que notre algorithme glouton ne soit pas "capable" de trouver une solution ne signifie pas qu'il n'existe pas de solution...en effet, il suffit de prendre 1 pièce de 5 cts et 3 pièces de 2 cts pour arriver à 11 cts. Recherchons un algorithme qui nous permettrait de trouver une solution optimale, quelle que soit la situation.

Afin de mettre au point un algorithme, essayons de trouver une relation de récurrence :

Soit X la somme à rendre, on notera $Nb(X)$ le nombre minimum de pièces à rendre. Nous allons nous poser la question suivante : Si je suis capable de rendre X avec $Nb(X)$ pièces, quelle somme suis-je capable de rendre avec $1 + Nb(X)$ pièces ?

Si j'ai à ma disposition la liste de pièces suivante : p_1, p_2, p_3, ..., p_n et que je suis capable de rendre X cts, je suis donc aussi capable de rendre :

- $X - p_1$
- $X - p_2$
- $X - p_3$
- ...
- $X - p_n$

(à condition que p_i (avec i compris entre 1 et n) soit inférieure ou égale à la somme restant à rendre)

(à condition que p_i (avec i compris entre 1 et n) soit inférieure ou égale à la somme restant à rendre)

Exemple : si je suis capable de rendre 72 cts et que j'ai à ma disposition des pièces de 2 cts, 5 cts, 10 cts, 50 cts et 1 euro, je peux aussi rendre :

- 72 - 2 = 70 cts
- 72 - 5 = 67 cts
- 72 - 10 = 62 cts
- 72 - 50 = 22 cts

Je ne peux pas utiliser de pièce de 1 euro.

Autrement dit, si $Nb(X - p_i)$ (avec i compris entre 1 et n) est le nombre minimal de pièces à rendre pour le montant $X - p_i$, alors $Nb(X) = 1 + Nb(X - p_i)$ est le nombre minimal de pièces à rendre pour un montant X. Nous avons donc la formule de récurrence suivante :

si $X = 0$: $Nb(X) = 0$

si $X > 0$: $Nb(X) = 1 + min(Nb(X - p_i))$ avec $1 \leq i < n$ et $p_i \leq X$

Le "min" présent dans la formule de récurrence exprime le fait que le nombre de pièces à rendre pour une somme $X - p_i$ doit être le plus petit possible.

À faire vous-même 3

Étudiez attentivement le programme Python suivant :

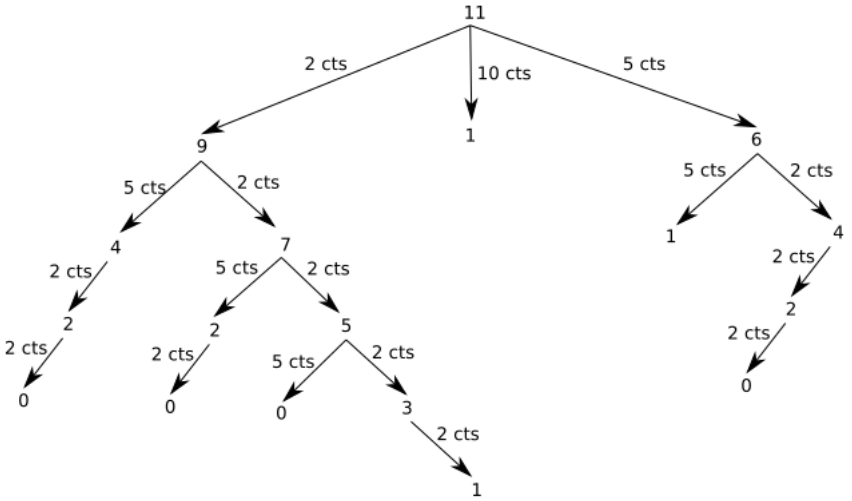
```
def rendu_monnaie_rec(P,X):
  if X==0:
    return 0
  else:
    mini = 1000
  for i in range(len(P)):
    if P[i]<=X:
      nb = 1 + rendu_monnaie_rec(P,X-P[i])
      if nb<mini:
        mini = nb
  return mini

pieces = (2,5,10,50,100)
```

Comme vous l'avez sans doute remarqué, pour être sûr de renvoyer le plus petit nombre de pièces, on attribue dans un premier temps la valeur 1000 à la variable mini (cette valeur 1000 est arbitraire, il faut juste une valeur suffisamment grande : on peut partir du principe que nous ne rencontrerons jamais de cas où il faudra rendre plus de 1000 pièces), ensuite, à chaque appel récursif, on "sauvegarde" le plus petit nombre de pièces dans cette variable mini.

À faire vous-même 4

Testez le programme du "À faire vous-même 3" en saisissant dans la console Python "rendu_monnaie_rec(pieces,11)" (on recherche le nombre minimum de pièces à rendre pour une somme de 11 cts. Les pièces disponibles sont : 2 cts, 5 cts, 10 cts, 50 cts, 100 cts (1 euro))

Le programme du "À faire vous-même 3" n'est pas des plus simple à comprendre (même si on retrouve bien la formule de récurrence définit un peu au-dessus), voici un schéma (avec une somme à rendre de 11 centimes) qui vous permettra de mieux comprendre le principe de cet algorithme :



Plusieurs remarques s'imposent :

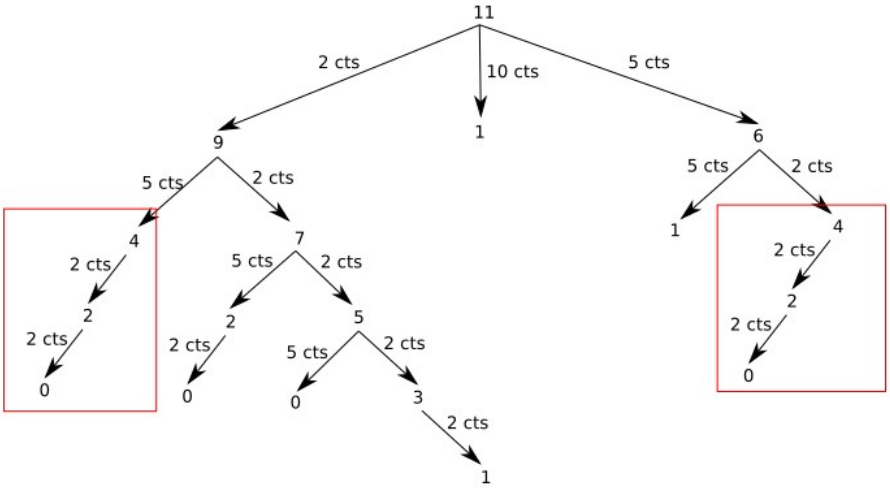
- comme vous pouvez le remarquer sur le schéma, tous les cas sont "traités" (quand un algorithme "traite" tous les cas possibles, on parle souvent de méthode "brute force").
- pour certains cas, on se retrouve dans une "impasse" (cas où on termine par un "1"), dans cette situation, la fonction renvoie "1000" ce qui permet de s'assurer que cette "solution" (qui n'en est pas une) ne sera pas "retenue".
- la profondeur minimum de l'arbre (avec une feuille 0) est de 4, la solution au problème est donc 4 (il existe plusieurs parcours : (5,2,2,2), (2,5,2,2)... qui donne à chaque fois 4)

À faire vous-même 5

Testez le programme du "À faire vous-même 3" en saisissant dans la console Python "rendu_monnaie_rec(pieces,171)" (on recherche le nombre minimum de pièces à rendre pour une somme de 1,71 euro. Les pièces disponibles sont : 2 cts, 5 cts, 10 cts, 50 cts, 100 cts (1 euro)).

Comme vous pouvez le constater le programme ne permet pas d'obtenir une solution, pourquoi ? Parce que les appels récursifs sont trop nombreux, on dépasse la capacité de la pile.

Comme vous avez peut-être déjà dû le remarquer, même dans le cas simple évoqué ci-dessus (11 cts à rendre), nous faisons plusieurs fois exactement le même calcul. Par exemple on retrouve 2 fois la branche qui part de "4" :



Il va donc être possible d'appliquer la même méthode que pour Fibonacci : la programmation dynamique.

À noter que dans des cas plus "difficiles à traiter" comme 1,71 euro, on va retrouver de nombreuses fois exactement les mêmes calculs, il est donc potentiellement intéressant d'utiliser la programmation dynamique.

À faire vous-même 6

Étudiez attentivement le programme Python suivant :

```
def rendu_monnaie_mem(P,X):
  mem = [0]*(X+1)
  return rendu_monnaie_mem_c(P,X,mem)

def rendu_monnaie_mem_c(P,X,m):
  if X==0:
    return 0
  elif m[X]>0:
    return m[X]
  else:
    mini = 1000
    for i in range(len(P)):
      if P[i]<=X:
        nb=1+rendu_monnaie_mem_c(P,X-P[i],m)
        if nb<mini:
          mini = nb
          m[X] = mini
  return mini

pieces = (2,5,10,50,100)
```

Ce programme ressemble beaucoup à programme utiliser pour la suite de Fibonacci, il ne devrait donc pas vous poser de problème.

À faire vous-même 7

Testez le programme du "À faire vous-même 6" en saisissant dans la console Python "rendu_monnaie_mem(pieces,171)" (on recherche le nombre minimum de pièces à rendre pour une somme de 1,71 euro. Les pièces disponibles sont : 2 cts, 5 cts, 10 cts, 50 cts, 100 cts (1 euro)).

Comme vous pouvez le constater, au contraire du cas précédent (programme du "À faire vous-même 3"), il suffit d'une fraction de seconde pour que le programme basé sur la programmation dynamique donne une réponse correcte.